

O artigo The practical Test pyramid, de Ham Vocke, apresenta a pirâmide de testes como uma forma prática de organizar os testes automatizados de um software. A ideia principal é utilizar testes com diferentes níveis de abrangência, mantendo muitas testes pequenos e rápidos na base da pirâmide e poucas testes amplos e lentos no topo. Essa organização ajuda a criar uma suíte de testes rápida, confiável e mais fácil de manter.

A automação possui papel fundamental nesse processo. Testar manualmente todas as alterações de um sistema é lento, repetitivo e sujeito a erros. Com testes automatizados, a equipe consegue descobrir rapidamente quando uma mudança quebra alguma funcionalidade, facilitando práticas como integração contínua e entrega contínua. A automação também permite modificar e refatorar o código com maior segurança, pois os testes fornecem feedback constante sobre o funcionamento da aplicação.

Na base da pirâmide ficam os testes unitários, que verificam pequenas partes do código de maneira isolada. Eles devem representar a maior parte da suíte porque são rápidos, simples e permitem identificar erros com precisão. O artigo recomenda testar principalmente o comportamento observável do código, evitando criar testes excessivamente dependentes de detalhes internos da implementação. Também podem ser utilizados mocks e stubs para substituir dependências externas e manter os testes rápidos. Uma estrutura recomendada é Arrange, Act, Assert, equivalente ao formato Given, When e Then utilizado em BDD.

Acima estão os testes de integração, utilizados para verificar se a aplicação se comunica corretamente com elementos externos, como bancos de dados, sistemas de arquivos, APIs e outros serviços. Eles são mais lentos que os testes unitários, mas são importantes porque verificam situações que não podem ser garantidas apenas testando componentes isolados. Em sistemas distribuídos também podem ser utilizados testes de contrato, que garantem que consumidores e provedores de uma API continuam respeitando os contratos combinados, reduzindo o risco de uma alteração em um serviço quebrar outro.

No topo ficam os testes de interface e testes end-to-end, que verificam o sistema de forma mais completa e próxima da experiência real do usuário. Apesar de proporcionarem confiança, esses testes são mais lentos, difíceis de manter e mais sujeitos a falhas ocasionais. Por isso, devem existir em quantidade reduzida e se concentrar apenas nos fluxos mais importantes do sistema. Casos extremos e detalhes de regras de negócio devem, sempre que possível, ser verificados nos níveis inferiores da pirâmide.

O artigo também destaca que a automação não elimina totalmente a necessidade de testes exploratórios. A análise manual ainda pode encontrar problemas de usabilidade, desempenho, mensagens de erro e situações que não foram previstas nos testes automatizados. Quando um defeito é encontrado, é interessante transformá-lo posteriormente em um teste automatizado para evitar regressões.

Por fim, uma boa estratégia deve evitar duplicação de testes. Sempre que possível, um componente deve ser testado no nível mais baixo da pirâmide capaz de fornecer confiança suficiente. Testes rápidos devem ser executados primeiro na pipeline e os mais lentos posteriormente. Além disso, o código dos testes deve receber o mesmo cuidado do código de produção, sendo simples, legível e fácil de manter. Dessa forma, a pirâmide de testes permite equilibrar velocidade, confiança e custo de manutenção, criando uma estratégia de testes automatizados mais eficiente.

Marcos do Rio Preto

2211254